



Sistema de Gestión de Incidencias

Alumna: Carla Isabel Blacio Reyes

Universidad Nacional Autónoma de México

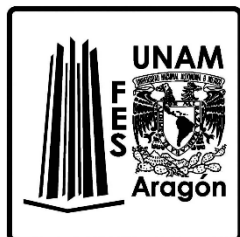
Facultad de Estudios Superiores Aragón

Vinculación Empresarial

Grupo: 2007

Profesor: Aaron Velasco Agustin

Edo. México a 29 de mayo del 2026



Índice

Índice	2
Resumen Ejecutivo	4
Introducción	5
Planteamiento del problema.....	6
Objetivos	7
Objetivo general	7
Objetivos específicos	7
Marco teórico	8
Alcance del sistema.....	9
¿Qué incluye? (In Scope)	9
¿Qué no incluye? (Out of Scope).....	9
Requerimientos del sistema.....	10
Requerimientos funcionales	10
Requerimientos no funcionales	12
Reglas de negocio.....	13
Casos de Uso	14
Diagrama General (Roles y Funciones)	14
Casos de Uso Específicos	15
CU-01 Crear Usuario.....	15
CU-02 Iniciar Sesión.....	16
CU-03 Registrar Incidencia	16
CU-04 Consultar Estado de Incidencias.....	17
CU-05 Asignar Incidencia.....	17
CU-06 Atender Incidencia	18
Historias de Usuario	19

Tecnologías Seleccionadas.....	20
Backend: PHP	20
Frontend: JavaScript + TailwindCSS	20
<i>Base de Datos</i> : MySQL.....	20
Contenedores: Docker + Docker Compose	20
Arquitectura del Sistema	21
Cliente (Frontend)	21
Servidor (Backend)	21
Base de Datos.....	21
Contenedores y Orquestación.....	21
Diagrama de arquitectura	22
Estructura del Proyecto y Distribución.....	22
Organización de carpetas.....	22
Uso de Contenedores y Despliegue.....	25
Dockerfile.....	25
docker-compose.....	25
Modelo de Base de Datos	26
Diagrama ER.....	27
Estrategia de Control de Versiones.....	28
Riesgos del Proyecto	29
Riesgos técnicos	29
Riesgos organizacionales	29
Conclusiones	30
Bibliografía.....	31

Resumen Ejecutivo

El proyecto Sistema de Gestión de Incidencias surge como respuesta a las deficiencias actuales en el control y seguimiento de reportes de mantenimiento en las instalaciones de la FES Aragón. La ausencia de un sistema formal ha generado pérdida de información, falta de métricas y desorganización en la asignación de tareas.

La solución propuesta consiste en el desarrollo de un sistema web centralizado que permita:

- Registrar incidencias.
- Dar seguimiento a cada reporte.
- Asignar responsables y controlar tiempos de atención.
- Generar métricas administrativas y reportes estratégicos.

Las tecnologías seleccionadas para la implementación son:

- Backend: PHP
- Frontend: TailwindCSS para diseño responsivo y moderno
- Base de datos: MySQL
- Contenedores y despliegue: Docker y Docker Compose

El beneficio esperado es optimizar la gestión administrativa del mantenimiento institucional, garantizando trazabilidad de los reportes, reducción de pérdidas de información y disponibilidad de métricas para la toma de decisiones.

El alcance general del sistema incluye la cobertura de incidencias en todas las áreas de la institución (aulas, baños, laboratorios, áreas comunes y edificios administrativos), con un panel administrativo para control de estados y un módulo de análisis para métricas estratégicas.

Introducción

El mantenimiento institucional es un proceso crítico para garantizar la operatividad y seguridad de las instalaciones de una organización. Sin embargo, en la FES Aragón se ha identificado que los reportes de incidencias se realizan de manera informal, lo que ocasiona pérdida de información, falta de seguimiento y ausencia de métricas confiables.

La importancia de contar con un sistema digital radica en la necesidad de centralizar la gestión de incidencias, optimizar la asignación de tareas y disponer de indicadores que permitan evaluar la eficiencia administrativa. La digitalización de este proceso no solo mejora la trazabilidad de los reportes, sino que también fortalece la toma de decisiones estratégicas al proporcionar datos precisos y oportunos.

El impacto administrativo esperado es significativo: reducción de tiempos de atención, mayor control en la asignación de responsables, disponibilidad de métricas para evaluar desempeño y una mejora sustancial en la organización de los procesos de mantenimiento.

Este trabajo se centrará en el diseño e implementación de un sistema web de gestión de incidencias, abordando desde el planteamiento del problema hasta la definición de objetivos, requerimientos, arquitectura y despliegue con tecnologías modernas como PHP, MySQL, TailwindCSS y Docker Compose. De esta manera, se busca demostrar cómo la integración de herramientas de desarrollo web y contenedores puede ofrecer una solución práctica, escalable y eficiente para la administración de reportes de mantenimiento.

Planteamiento del problema

En la situación actual, la FES Aragón enfrenta serias limitaciones en el control y seguimiento de los reportes de mantenimiento de sus instalaciones (aulas, baños, laboratorios, áreas comunes y edificios administrativos). Los procesos se realizan de manera informal, lo que genera una serie de problemas operativos:

- Pérdida de información: los reportes no quedan registrados en un sistema centralizado, dificultando la trazabilidad.
- Falta de seguimiento: no existe un mecanismo que permita verificar el estado de cada incidencia ni el cumplimiento de tiempos de atención.
- Ausencia de métricas: la administración carece de indicadores que permitan evaluar la eficiencia del área de mantenimiento.
- Desorganización en la asignación de tareas: los responsables se asignan de manera manual y sin control estructurado.
- Impacto operativo: estas deficiencias provocan retrasos en la atención, duplicidad de esfuerzos y baja calidad en el servicio de mantenimiento.

Los riesgos derivados de esta situación incluyen:

Riesgo técnico: imposibilidad de escalar el proceso de gestión de incidencias conforme aumente la demanda.

Riesgo organizacional: pérdida de confianza en la administración por parte de usuarios y personal técnico, además de un impacto negativo en la eficiencia institucional.

En consecuencia, se requiere un sistema web que formalice el proceso de gestión de incidencias en la FES Aragón, garantice la trazabilidad de la información y proporcione herramientas de análisis para la toma de decisiones estratégicas.

Objetivos

Objetivo general

Desarrollar un sistema web centralizado para la FES Aragón que permita la gestión administrativa de incidencias de mantenimiento institucional, integrando módulos especializados (login, registro, panel y seguimiento) y un servicio de base de datos, con el fin de garantizar trazabilidad, eficiencia operativa y generación de métricas estratégicas.

Objetivos específicos

- Implementar un módulo de login que gestione la autenticación de usuarios y el acceso seguro al sistema.
- Desarrollar un módulo de registro de incidencias que permita capturar datos de manera estructurada, incluyendo ubicación, descripción y clasificación del reporte.
- Diseñar un panel administrativo que facilite el seguimiento de cada incidencia, con estados definidos, asignación de responsables y control de tiempos de atención.
- Construir un módulo de seguimiento que permita visualizar el avance de los reportes, registrar notas técnicas y cerrar incidencias con trazabilidad completa.
- Integrar un servicio de base de datos MySQL para almacenar de forma persistente los reportes, usuarios, roles y métricas.

Marco teórico

El desarrollo de un sistema web de gestión de incidencias institucional requiere comprender diversos conceptos técnicos y administrativos que fundamentan su diseño e implementación:

Sistemas de Información: Son herramientas que permiten recopilar, procesar y administrar datos para apoyar la toma de decisiones. En el contexto de la FES Aragón, el sistema de gestión de incidencias se convierte en un sistema de información orientado al mantenimiento institucional.

Sistemas Administrativos: Se enfocan en la organización y control de procesos internos. El sistema propuesto busca mejorar la administración de reportes de mantenimiento, asignación de responsables y seguimiento de tareas.

Desarrollo Web: Involucra la creación de aplicaciones accesibles desde navegadores, utilizando tecnologías como PHP para la lógica de negocio y TailwindCSS para la interfaz gráfica responsiva y moderna.

Bases de Datos: Constituyen el núcleo de almacenamiento de información. En este proyecto se utiliza MySQL, que permite estructurar y consultar datos relacionados con usuarios, reportes, estados y métricas.

Gestión de Incidencias: Se refiere al proceso de registrar, clasificar y dar seguimiento a problemas detectados en las instalaciones. El sistema digitaliza este proceso, garantizando trazabilidad y métricas confiables.

Git: Herramienta de control de versiones que facilita el trabajo colaborativo, el manejo de ramas y la integración de cambios en el código.

Docker y Docker Compose: Tecnologías de contenedores que permiten empaquetar y desplegar aplicaciones de manera portable y escalable. En este proyecto se utilizan para definir y ejecutar los módulos (login, registro, panel, seguimiento) junto con el servicio de base de datos.

Alcance del sistema

¿Qué incluye? (In Scope)

- Login y registro de nuevos usuarios para garantizar acceso seguro y controlado.
- Formulario de nuevo reporte con captura de datos estructurados.
- Panel de evaluación y control administrativo para dar seguimiento a incidencias.
- Captura y almacenamiento de datos en la base de datos MySQL, asegurando persistencia y trazabilidad.

¿Qué no incluye? (Out of Scope)

- Módulo de inventarios para materiales o recursos físicos.
- Notificación automática de seguimiento vía correo electrónico o mensajería.
- Gestión de recursos humanos (control de personal, nómina o asistencia).
- Aplicación móvil nativa para Android o iOS.
- Integración con sensores IoT para monitoreo automatizado de instalaciones.

Requerimientos del sistema

Requerimientos funcionales

Tabla 1: Requerimientos funcionales

ID	Requerimiento
RF-01	El sistema deberá permitir al usuario seleccionar el tipo de ubicación (Aulas, Baños, Laboratorios, etc.) desde una lista desplegable precargada.
RF-02	El sistema deberá incluir un campo de texto enriquecido para que el usuario describa el problema con detalle y permita un mínimo de 10 y un máximo de 500 caracteres.
RF-03	El sistema deberá permitir al usuario asignar un nivel de prioridad al reporte (ej. Baja, Media, Alta) mediante botones de opción.
RF-04	El sistema deberá permitir al usuario seleccionar el turno en que ocurrió el problema (Matutino, Vespertino, Nocturno).
RF-05	El sistema deberá generar un identificador único de forma automática para cada nuevo reporte registrado con éxito.
RF-06	El sistema deberá mostrar una pantalla de confirmación al usuario.
RF-07	El sistema deberá validar al enviar el reporte que todos los campos obligatorios estén completos antes de mandarlo.
RF-08	El sistema deberá capturar automáticamente la fecha y hora exacta en que se crea el reporte, almacenándola en la base de datos.

RF-09	El sistema deberá contar con un login que valide las credenciales de los usuarios y sus permisos antes de permitir el acceso a las funcionalidades.
RF-10	El sistema deberá permitir la creación de nuevos usuarios y asignación de roles que definan el nivel de acceso dentro de la plataforma.
RF-11	El sistema deberá mantener la sesión activa del usuario durante la navegación entre los diferentes módulos (login, registro, panel, seguimiento), evitando la necesidad de iniciar sesión en cada uno de ellos.
RF-12	El sistema deberá permitir la visualización de la información de los reportes creados.
RF-13	El sistema deberá permitir solo a los usuarios asignados finalizar la tarea.

Nota. Fuente: Elaboración propia

Requerimientos no funcionales

Tabla 2: Requerimientos no funcionales

ID	Requerimiento
RNF-01	El módulo de registro de incidencias deberá tener una disponibilidad mínima del 99.5% durante el horario laboral.
RNF-02	El sistema deberá garantizar que ningún reporte se pierda en caso de error de conexión, asegurando mecanismos de recuperación.
RNF-03	La interfaz del formulario de reporte deberá ser intuitiva y responsiva, permitiendo su correcto uso desde dispositivos móviles y equipos de escritorio.
RNF-04	El formulario deberá ser completamente funcional en los principales navegadores web.
RNF-05	El tiempo de respuesta del sistema para el guardado de un reporte no deberá exceder los 3 segundos en condiciones normales de operación.
RNF-06	El equipo de soporte deberá responder incidencias técnicas en un máximo de 24 horas hábiles.
RNF-07	El código fuente del módulo deberá estar estructurado siguiendo un patrón de diseño definido, facilitando futuras modificaciones y mantenibilidad.
RNF-08	El sistema deberá funcionar en contenedores bajo la estructura de microservicios, para garantizar portabilidad y escalabilidad.

Nota. Fuente: Elaboración propia

Reglas de negocio

Tabla 3: Reglas de negocio

ID	Requerimiento
RB-01	Las incidencias deben contar con un nivel de prioridad (Alta, Media, Baja) con base en su impacto y urgencia, con el fin de establecer un orden de atención.
RB-02	Las incidencias únicamente podrán ser atendidas por el usuario asignado, quien será responsable de dar seguimiento y registrar la resolución dentro del sistema.
RB-03	La asignación de incidencias solo podrá ser realizada por un usuario con rol de administrador dentro del sistema.
RB-04	El sistema únicamente permitirá registrar incidencias que cuenten con todos los datos obligatorios (tipo de ubicación, descripción del problema, turno, prioridad y persona que reporta), garantizando la información necesaria para su atención.
RB-05	Cada incidencia deberá contar con un estatus definido (pendiente y/o atendido), el cual se actualizará conforme avance el proceso de atención.
RB-06	El sistema deberá registrar automáticamente la fecha y hora exacta en que se crea cada incidencia, asegurando trazabilidad temporal.

Nota. Fuente: Elaboración propia

Casos de Uso

Diagrama General (Roles y Funciones)

- Administrador

Crear usuario

Iniciar sesión

Generar reporte de incidencia

Consultar estado de incidencias

Asignar incidencias

- Usuario

Crear usuario

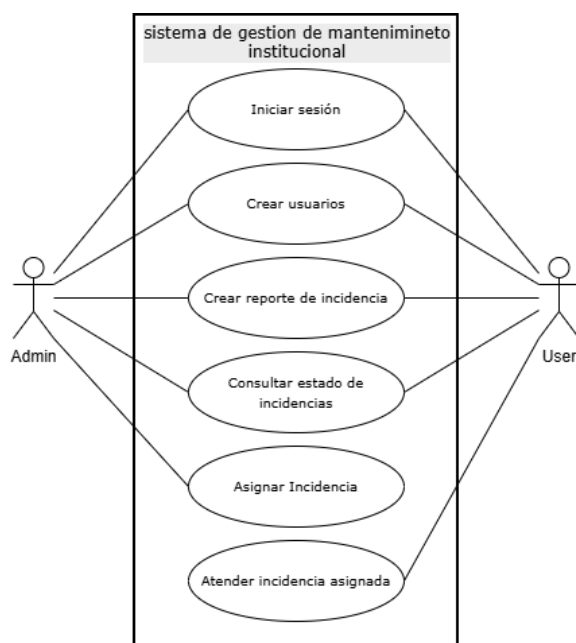
Iniciar sesión

Generar reporte de incidencia

Consultar estado de incidencias

Atender incidencia asignada (el sistema cambia automáticamente el estado a “atendida”)

Figura 1: diagrama de caso de uso general



Nota. Fuente: Elaboración Propia en Draw.io. (s.f.). <https://www.drawio.com/>

Casos de Uso Específicos

CU-01 Crear Usuario

Actor: Administrador / Usuario

Descripción: Permite a cualquier persona crear una cuenta dentro del sistema SGMI para acceder a las funcionalidades disponibles según su rol. El proceso incluye el registro de correo electrónico, contraseña y selección de rol (Administrador o Usuario).

Precondición: El sistema debe estar disponible y el actor debe acceder correctamente al módulo de registro.

Flujo principal: El actor accede al formulario de registro → ingresa correo y contraseña → confirma contraseña → elige un rol → el sistema valida y crea la cuenta.

Flujos alternos:

- Si el correo ya existe en la base de datos, el sistema mostrará un mensaje indicando que el usuario ya está registrado.
- Si las contraseñas no coinciden, el sistema solicitará corregir la confirmación antes de continuar.
- Si algún campo obligatorio está vacío, el sistema impedirá el registro y mostrará una alerta.

Postcondición: El sistema almacena los datos del nuevo usuario en la base de datos y te redirige a la pantalla de Login.

CU-02 Iniciar Sesión

Actor: Administrador / Usuario

Descripción: Permite a los actores acceder al sistema SGMI mediante la validación de sus credenciales, habilitando el uso de los módulos correspondientes según su rol.

Precondición: El usuario debe contar con una cuenta previamente registrada en el sistema.

Flujo principal: El actor ingresa credenciales → el sistema valida permisos → se habilita acceso al panel y módulos correspondientes.

Flujos alternos:

- Si las credenciales son incorrectas, el sistema muestra un mensaje de error e impide el acceso.
- Si algún campo obligatorio está vacío, el sistema solicita completarlo antes de continuar.

Postcondición: El actor accede al sistema con su sesión activa y puede utilizar las funcionalidades disponibles según su rol (Administrador o Usuario).

CU-03 Registrar Incidencia

Actor: Administrador / Usuario

Descripción: Permite a los actores registrar una nueva incidencia en el sistema SGMI, capturando información detallada sobre el problema detectado en las instalaciones.

Precondición: El actor debe haber iniciado sesión en el sistema y tener acceso al módulo de registro de incidencias.

Flujo principal: El actor llena el formulario (tipo de ubicación, descripción, turno, prioridad) → envía reporte → pide confirmación → el sistema genera identificador único y registra fecha/hora.

Flujos alternos:

- Si algún campo obligatorio está vacío, el sistema muestra un mensaje de error e impide el registro.
- Antes de enviar el reporte muestra una pantalla de confirmación.

Postcondición: El sistema almacena la incidencia en la base de datos con todos los datos capturados.

CU-04 Consultar Estado de Incidencias

Actor: Administrador / Usuario

Descripción: Permite a los actores visualizar el estado y detalles de las incidencias registradas en el sistema.

Precondición: El actor debe haber iniciado sesión en el sistema.

Flujo principal: El actor accede al panel → selecciona reporte → visualiza detalles (estatus, prioridad, fecha, responsable asignado).

Flujos alternos:

- Si no existen incidencias registradas, el sistema indican que no hay reportes disponibles.

Postcondición: El actor obtiene información actualizada sobre las incidencias registradas en el sistema.

CU-05 Asignar Incidencia

Actor: Administrador

Descripción: Permite al administrador asignar una incidencia pendiente a un usuario específico para su atención.

Precondición: El administrador debe haber iniciado sesión y existir incidencias pendientes en el sistema.

Flujo principal: El administrador consulta incidencias pendientes → selecciona reporte → asigna un usuario → el sistema guarda la asignación.

Flujos alternos:

- Si no hay incidencias pendientes, el sistema indica que no existen reportes por asignar.

Postcondición: La incidencia queda asignada a un usuario específico y registrada en el sistema.

CU-06 Atender Incidencia

Actor: Usuario

Descripción: Permite al usuario asignado atender la incidencia registrada, marcándola como atendida y generando el cambio automático de estado.

Precondición: El usuario debe haber iniciado sesión y tener incidencias asignadas en el sistema.

Flujo principal: El usuario consulta incidencias asignadas → atiende la incidencia → el sistema cambia automáticamente el estado a “atendida”.

Flujos alternos:

- Si el usuario intenta atender una incidencia que no le fue asignada, el sistema no muestra la opción de marcar como atendida.
- Si el reporte ya está marcado como atendido, el sistema impide duplicar la acción.

Postcondición: La incidencia queda registrada como atendida en el sistema, con fecha y hora de atención.

Historias de Usuario

Registro de Usuario

Como usuario o administrador, quiero poder crear una cuenta en el sistema para acceder a las funcionalidades según mi rol.

Inicio de Sesión

Como usuario o administrador, quiero poder iniciar sesión con mis credenciales para utilizar el sistema de manera segura y personalizada.

Registro de Incidencia

Como usuario o administrador, quiero poder registrar una incidencia mediante un formulario para reportar problemas en las instalaciones de manera estructurada.

Consulta de Incidencias

Como usuario o administrador, quiero poder consultar el estado de las incidencias registradas para dar seguimiento y conocer su avance.

Asignación de Incidencias

Como administrador, quiero poder asignar incidencias a un usuario específico para asegurar que cada reporte tenga un responsable claro.

Atención de Incidencias

Como usuario asignado, quiero poder marcar una incidencia como atendida para cerrar el ciclo de atención y que el sistema actualice su estado automáticamente.

Tecnologías Seleccionadas

El sistema SGMI se construyó utilizando un conjunto de tecnologías que garantizan portabilidad, escalabilidad y facilidad de mantenimiento:

Backend: PHP

Lenguaje de programación utilizado para la lógica del servidor.

Permite gestionar formularios, sesiones y conexión con la base de datos.

Justificación: amplia comunidad, soporte estable y facilidad de integración con MySQL.

Frontend: JavaScript + TailwindCSS

JavaScript: lenguaje de programación para la interacción dinámica en el cliente.

Justificación: permite validar formularios, mejorar la experiencia de usuario y manejar eventos en tiempo real.

TailwindCSS: framework CSS para diseño responsivo y moderno.

Justificación: facilita la construcción de interfaces intuitivas y adaptables a dispositivos móviles.

Base de Datos: MySQL

Motor de base de datos relacional para almacenar incidencias, usuarios y reportes.

Justificación: robustez, consultas rápidas y compatibilidad con PHP.

Contenedores: Docker + Docker Compose

Herramientas de virtualización para empaquetar y desplegar el sistema en contenedores.

Justificación: aseguran portabilidad, escalabilidad y facilidad de despliegue en distintos entornos.

Arquitectura del Sistema

El sistema SGAMI sigue una arquitectura cliente-servidor basada en contenedores y microservicios, lo que garantiza escalabilidad, mantenibilidad y portabilidad.

Componentes principales:

Cliente (Frontend)

Construido con JavaScript y TailwindCSS.

Se encarga de la interacción con el usuario: formularios, validaciones y visualización de datos.

Comunicación con el backend mediante peticiones HTTP/REST.

Servidor (Backend)

Desarrollado en PHP.

Contiene la lógica de negocio: registro de incidencias, asignación, autenticación y generación de reportes.

Expone servicios a través de una API REST.

Base de Datos

Implementada en MySQL.

Almacena usuarios, incidencias, estados y reportes.

Garantiza integridad y consistencia de la información.

Contenedores y Orquestación

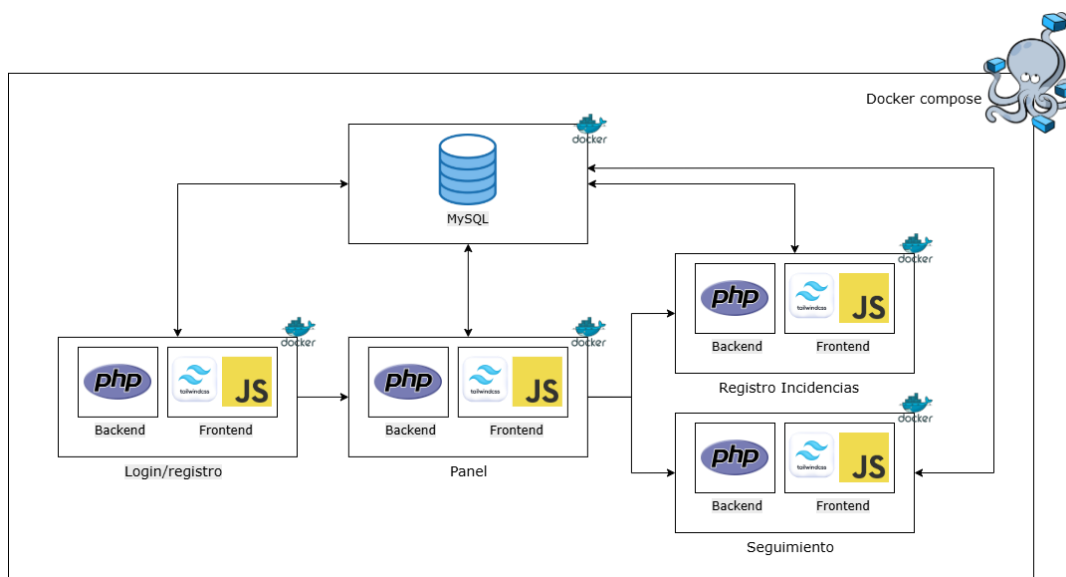
Docker + Docker Compose para empaquetar y desplegar los servicios.

Cada módulo (frontend, backend, base de datos) corre en un contenedor independiente.

Facilita la portabilidad y despliegue en distintos entornos.

Diagrama de arquitectura

Figura 2: Diagrama de arquitectura



Nota. Fuente: Elaboración Propia en Draw.io. (s.f.). <https://www.drawio.com/>

Estructura del Proyecto y Distribución

Organización de carpetas

El sistema SGMI se organiza en una estructura modular que facilita la mantenibilidad, escalabilidad y despliegue mediante contenedores Docker. Cada módulo representa una funcionalidad independiente del sistema y se ejecuta dentro de su propio servicio.

Los módulos principales, el proyecto incluye carpetas y archivos de soporte:

login/ → Módulo de autenticación de usuarios y para la creación de cuentas.

registro/ → Módulo para el registro de incidencias.

panel/ → Módulo administrativo para visualizar y gestionar reportes.

seguimiento/ → Módulo de seguimiento de incidencias para asignarlas y/o marcarlas como atendidas.

mysql_data/ → Carpeta que contiene los datos persistentes de la base de datos MySQL.

Dockerfile → Define la imagen base de PHP con Apache y las extensiones necesarias.

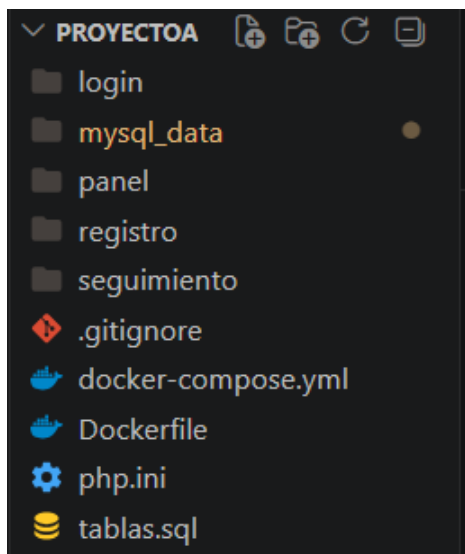
docker-compose.yml → Orquesta los servicios y contenedores del sistema.

php.ini → Archivo de configuración del entorno PHP.

tablas.sql → Script SQL con la estructura inicial de la base de datos.

.gitignore → Define los archivos y carpetas que no deben incluirse en el control de versiones.

Figura 3: Estructura del proyecto

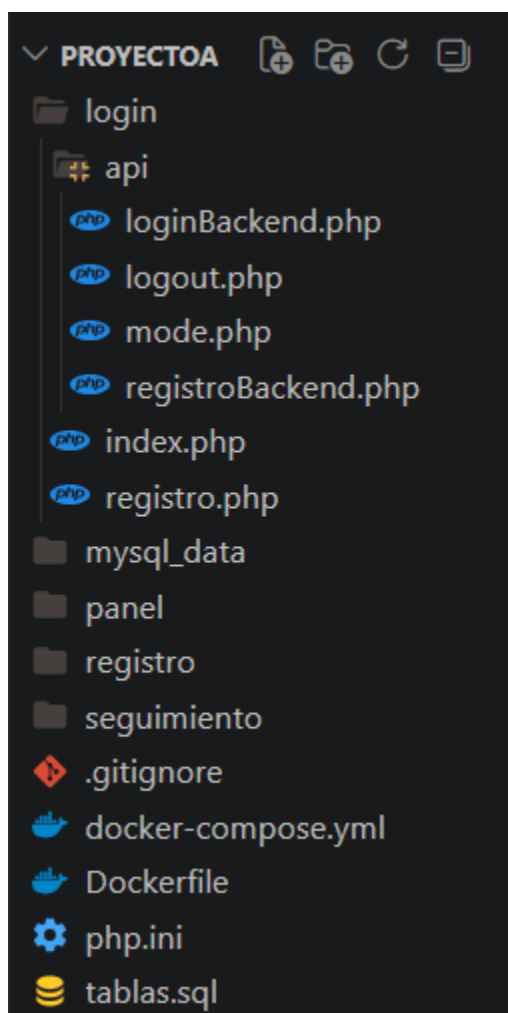


Nota. Fuente: Elaboración Propia en Visual Studio Code

Además, dentro de cada módulo, la organización de archivos sigue una lógica que hace referencia al modelo vista–controlador:

- En la carpeta api/ se encuentran los archivos PHP correspondientes al backend, encargados de la lógica de negocio y la interacción con la base de datos.
- En la raíz del módulo se ubican los archivos del frontend, responsables de la interfaz de usuario y la presentación de la información.

Figura 4: Estructura del proyecto por modulo



Nota. Fuente: Elaboración Propia en Visual Studio Code

Uso de Contenedores y Despliegue

Para garantizar la portabilidad y la facilidad de despliegue del sistema SGMI, se implementó una estrategia basada en contenedores Docker. Esta decisión permitió encapsular cada módulo del proyecto (login, registro, panel y seguimiento) junto con sus dependencias, asegurando que el sistema pudiera ejecutarse de manera uniforme en cualquier entorno.

El flujo de trabajo se apoyó en dos componentes:

Dockerfile

Donde se definió la imagen base de PHP con Apache y se instalaron las extensiones necesarias (mysqli, pdo, pdo_mysql) para la conexión con la base de datos.

docker-compose

Empleado para orquestar los servicios del sistema, incluyendo los módulos web y la base de datos MySQL.

Cada módulo se ejecutó como un servicio independiente dentro de su propio contenedor, con puertos específicos para su acceso (8081 para login, 8082 para panel, 8083 para registro y 8084 para seguimiento). El servicio de base de datos se desplegó en un contenedor propio (mysql), con persistencia de datos mediante volúmenes.

Este enfoque permitió:

- Portabilidad, al ejecutar el sistema en cualquier máquina con Docker instalado.
- Escalabilidad, al aislar cada módulo en su propio contenedor.
- Mantenibilidad, al centralizar la configuración en el archivo docker-compose.yml.
- Compatibilidad, al garantizar que las extensiones de PHP necesarias estuvieran disponibles en todos los entornos gracias al Dockerfile.

Modelo de Base de Datos

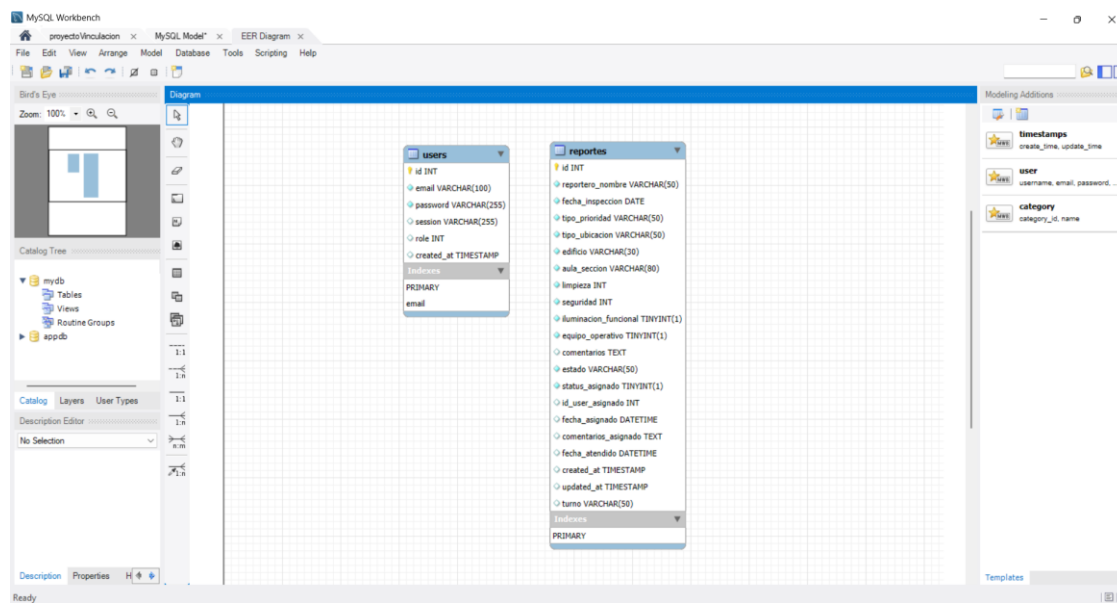
Tabla 3: Entidades principales de la DB

Entidad	Descripción	Campos
users	Almacena la información de los usuarios del sistema (administradores y usuarios).	id, email, password, session, role, created_at.
reportes	Registra los reportes generados por los usuarios.	reportero_nombre, fecha_inspección, tipo_prioridad, turno, tipo_ubicacion, edificio, aula_seccion, limpieza, seguridad, iluminacion_funcional, equipo_operativo, comentarios, estado,status_asignado, id_user_asignado, fecha_asignado, comentarios_asignado, fecha_atendido,created_at, updated_at

Nota. Fuente: Elaboración propia

Diagrama ER

Figura 5: Diagrama ER



Nota. Fuente: Elaboración Propia en MySQL Workbench

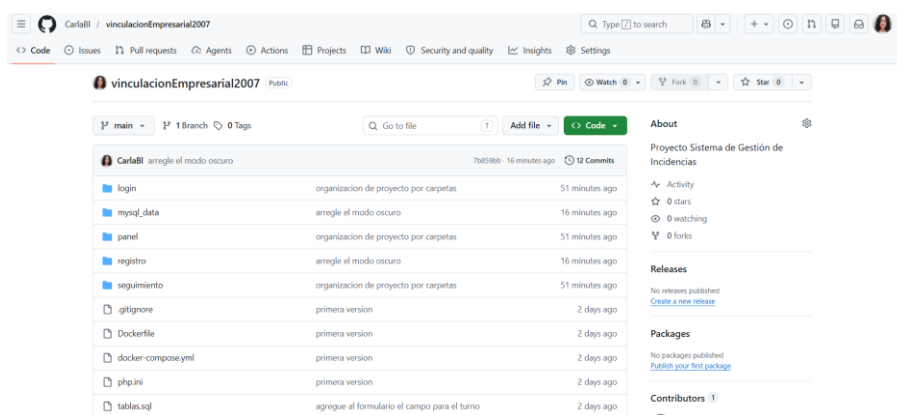
Estrategia de Control de Versiones

Durante el desarrollo del sistema SGMI se implementó un control de versiones utilizando Git como herramienta principal y GitHub como plataforma de alojamiento remoto. Esto permitió mantener un historial claro de cambios y asegurar la trazabilidad del proyecto.

El flujo de trabajo se basó principalmente en la rama principal (main), donde se fueron registrando los avances mediante commits periódicos y descriptivos. Cada commit reflejó una mejora, corrección o nueva funcionalidad, lo que facilitó identificar el progreso y posibles puntos de retroceso en caso de errores.

Aunque la mayor parte del trabajo se realizó directamente en la rama main, se aplicaron prácticas que simulan un flujo de ramas más avanzado. Por ejemplo, en ciertos momentos se creó una rama secundaria (login) para trabajar en el módulo de autenticación de manera aislada. Una vez completada y probada la funcionalidad, esta rama se integró nuevamente en la rama principal (main) mediante un merge, asegurando que el código estable se mantuviera limpio y sin interrupciones. Y posteriormente se borró la rama.

Figura 6: Repositorio de GitHub



Nota. Fuente CarlaBl. (s.f.). GitHub - CarlaBl/VinculacionEmpresarial2007: Proyecto Sistema de Gestión de incidencias. GitHub.

<https://github.com/CarlaBl/vinculacionEmpresarial2007>

Riesgos del Proyecto

Riesgos técnicos

En el proyecto los riesgos técnicos se relacionan principalmente con la dependencia de tecnologías específicas, ya que el sistema se construyó sobre PHP, MySQL y Docker, lo que implica que cualquier incompatibilidad futura en versiones podría afectar la estabilidad. También se identificó que las credenciales de acceso, como las contraseñas, se almacenan en texto plano, lo que representa un riesgo de seguridad en caso de accesos no autorizados. Otro aspecto técnico es la integridad de los datos, pues al no existir relaciones foráneas en la base de datos se corre el riesgo de inconsistencias entre usuarios y reportes. Además, la disponibilidad de los contenedores depende de la correcta configuración de docker-compose y de la persistencia de volúmenes, lo que podría ocasionar pérdida temporal de servicios o información. Finalmente, se reconoce que la escalabilidad del sistema es limitada, ya que la arquitectura modular funciona bien en entornos pequeños, pero requeriría ajustes para soportar un mayor número de usuarios o reportes.

Riesgos organizacionales

En cuanto a los riesgos organizacionales, el proyecto enfrenta la posibilidad de que los usuarios finales tengan dificultades para adaptarse al sistema si no se proporciona capacitación adecuada. Existe también el riesgo de resistencia al cambio, ya que la transición de procesos manuales a un sistema digital puede generar rechazo en algunos miembros de la institución. Otro punto crítico es la dependencia de un equipo reducido para el mantenimiento, lo que podría comprometer la continuidad del proyecto ante ausencias o rotación de personal. Asimismo, la gestión de incidencias requiere políticas claras de seguimiento y atención, pues de lo contrario el sistema podría ser subutilizado o no reflejar la realidad operativa. Finalmente, la comunicación interna entre áreas administrativas y técnicas es un factor clave, ya que la falta de coordinación puede retrasar la implementación de mejoras o correcciones.

Conclusiones

En el proyecto SGMI se logró implementar un sistema modular que integra de manera coherente los procesos de autenticación, registro, gestión y seguimiento de incidencias. El uso de contenedores Docker permitió garantizar la portabilidad y la facilidad de despliegue, mientras que la organización del código en módulos independientes favoreció la mantenibilidad y la escalabilidad del sistema.

A nivel técnico, se consolidó una arquitectura sencilla pero funcional, capaz de responder a las necesidades básicas de gestión de incidencias. La integración de Git y GitHub como herramientas de control de versiones aseguró la trazabilidad de los cambios y la estabilidad del código, mientras que el uso de MySQL como motor de base de datos proporcionó un entorno confiable para el almacenamiento de la información.

En el ámbito organizacional, el proyecto representa un avance significativo hacia la digitalización de procesos, reduciendo la dependencia de registros manuales y mejorando la eficiencia en la atención de incidencias. Asimismo, se identificaron riesgos técnicos y organizacionales que servirán como base para futuras mejoras, especialmente en temas de seguridad, integridad de datos y capacitación de usuarios.

Finalmente, el desarrollo del sistema SGMI permitió adquirir experiencia práctica en el uso de tecnologías modernas como Docker, PHP y Git, además de fortalecer competencias en diseño de arquitecturas modulares y documentación técnica. El proyecto constituye un punto de partida sólido para futuras versiones más robustas, donde se podrán incorporar nuevas funcionalidades, optimizar la base de datos y ampliar la cobertura del sistema en entornos más complejos.

Bibliografía

CarlaBl. (s. f.). GitHub - CarlaBl/VinculacionEmpresarial2007: Proyecto Sistema de Gestión de incidencias. GitHub. <https://github.com/CarlaBl/vinculacionEmpresarial2007>

Documentation - Tailwind CSS. (2021, 17 junio). <https://v2.tailwindcss.com/docs>
Draw.io. (s. f.). <https://www.drawio.com/> Inc, D. (2026, 28 mayo).

Docker compose. Docker Documentation. <https://docs.docker.com/compose/> Llamas, L. (2024, 18 octubre).

Cheatsheet de PHP. Luis Llamas. <https://www.luisllamas.es/en/cheatsheet-php/>

Git. (s. f.). <https://git-scm.com/>